COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	10/13/2026	**DATE SUBMITTED:**	10/13/2026
NAME:	Jose Martin C. Rodulfa Dominic Rodnee A. Panti	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 12.1 **Algorithmic Strategies**

1. PROCEDURE OUTPUT

Strategy	Algorithm	Analysis
Recursion	Linked List	A node calls itself inside it which points to another node.
Brute Force	Bubble Sort	It requires the code to check each individual value in an index before arranging.
Backtracking	Binary Search Tree	It searches through each child of the tree, backtracking back to the head when it finds no more children.
Greedy	Change-Making	It uses a method to optimally reduce the number of coins needed for change of a specific amount.
Divide-and-Conquer	Merge Sort	It divides the array into individual parts then rearranges them back by pairs towards a sequential arrangement.

TABLE 12-1. Algorithmic Strategies and Examples

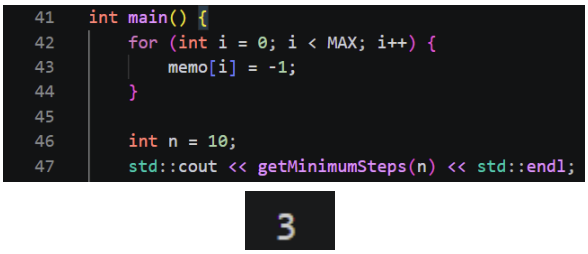
Memoization Output:	
Analysis:	Utilizing the given function, we can see that the Memoization approach breaks down “n” into what we call “subproblems” (as seen in the modulo operations). Each output from this operation is stored in “memo[n]”, a soft catch that allows all results to be stored, preventing unnecessary reiterations. It has a time complexity of O(n) and a space complexity of O(n) as well, as it depends on the input “n” as to how much the function recurses.

TABLE 12-2. Memoization Implementation

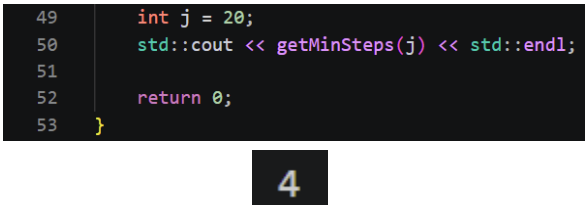
BUDP Output:	
Analysis:	With respect to the provided function above, this code assesses the target “n” one at a time (dp[1]), checking each condition, storing each result, and then categorizing them via a DP table. This is another method for minimizing unnecessary recursions, instead utilizing function reiteration to evaluate the target.

TABLE 12-3. Bottom-up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```
Target Cost: 25
Number of Paths: 0
```

suppleRODULFA.cpp > ...

```cpp
#include <iostream>
int countPaths(int matrix[][4], int row, int col, int cost)
{
    if (cost < 0)
    {
        return 0;
    }
    if (row == 0 && col == 0)
    {
        return (cost == matrix[0][0]) ? 1 : 0;
    }
    if (row == 0)
    {
        return countPaths(
            matrix,
            0,
            col - 1,
            cost - matrix[row][col]
        );
    }
    if (col == 0)
    {
        return countPaths(
            matrix,
            row - 1,
            0,
            cost - matrix[row][col]
        );
    }
    return countPaths(
                matrix,
                row - 1,
                col,
                cost - matrix[row][col]
            )
            +
```

```
            countPaths(
                matrix,
                row,
                col - 1,
                cost - matrix[row][col]
            );
}

int main()
{
    int matrix[4][4] =
    {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };
    int cost = 25;
    std::cout << "Target Cost: "
              << cost << std::endl;
    std::cout << "Number of Paths: "
              << countPaths(matrix, 3, 3, cost)
              << std::endl;
    return 0;
}
```

Rodulfa

Panti

CONCLUSION:

Rodulfa: Through this hands-on activity, I learned how different algorithmic strategies work and how they can be applied to solve problems more efficiently. I understood how recursion, brute force, backtracking, greedy algorithms, and divide-and-conquer differ in their approaches. I also learned how memoization and bottom-up dynamic programming can reduce unnecessary computations by storing and reusing results. Overall, this activity helped me better understand how choosing the right algorithm can improve a program's efficiency and performance.

Panti:

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: N/A

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: N/A

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: N/A

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.